

Stage 2 — Capstone Marking Guide

Ubuntu Bridge Initiative · Cybersecurity Internship · Cohort 1

Confidential — for graders only. Do not share with interns.

Welcome — read this first

Thank you for grading Stage 2. This guide gives you the **answer to every deliverable** and tells you exactly what to mark, in what order, and how. You do **not** need to be a pentester. If you can read an intern's report against the answer key below, check that a claim is tied to a real line of evidence, and tell whether a CVSS vector is defended or just pasted, you can grade this stage well.

Stage 2 is **Web Application Security — the attack surface**. It is built so that someone who only pasted an AI answer cannot pass: most of the marks are for *proving findings from the evidence pack and prioritising them like an engineer*, not for naming a vulnerability.

If you are ever unsure whether an answer is correct, **flag it for super-admin review** rather than guess.

The story Stage 2 is testing

The threat actor — **The Griot** — reached **Sankofa Digital** (a Nigerian fintech) through `sankofa.internal/legacy-admin/`; a Django/PHP admin tool from 2019 that should have been decommissioned, still internet-reachable. The intern's job is to do the post-incident pentest: prove every weakness from the recovered evidence, rank it by business risk, and brief engineering and the board.

The recurring lesson: **the vulnerabilities chain**. No single bug is the story — the SQL injection, the broken auth tokens, the file-read, and the export feature combine into one path from "internet-reachable login page" to "312 customer records exfiltrated."

The named staff the intern writes for:

- **Bayo Ogunyemi** — Head of Engineering. The report is *for him*; he acts on risk-prioritised findings.
- **Amaka Eze** — Head of Security. Co-signs the report.
- **Ngozi Ojukwu** — DevSecOps Lead. Owns the fixes.
- **Tunde Afolabi** — Threat Intel. Provided the HTTP capture.

How Stage 2 is structured — lab (auto) + capstone (you hand-grade)

Stage 2 has two halves. The **lab** is eight in-platform tasks, all **auto-graded by the platform**. The **capstone** is four written deliverables (D1–D4) that **you grade by hand**. *You only hand-grade the capstone*. The lab answer key further down is for context only — the capstone refers back to it.

Part	Item	Pts	Who grades
Lab	Tasks 1–8 (flags + MCQ)	auto	**Platform**
Capstone	**D1 — Findings catalogue (+ the decoy)**	**30**	**You**
Capstone	**D2 — Exploit chain + CVSS & business impact**	**35**	**You**
Capstone	**D3 — The report Bayo acts on (+ detection stopgaps)**	**40**	**You**
Capstone	**D4 — Ethics stance**	**15**	**You**
	Capstone total (what you score)	**120**	you

The lab is auto-scored and weighted in separately by programme staff. **Your number is the capstone score out of 120, recorded as a percentage** ($\text{points earned} \div 120 \times 100$). The advance/eliminate cutoff is set by programme staff in the promotion flow — your job is an accurate score, not to decide who passes. Grade honestly and leave a one-line justification on every deliverable; an intern may be eliminated on your number.

How to grade — step by step

Total time per submission: 30–40 minutes. Take a break between submissions; the third in a row gets harsher than the first.

- 1. Confirm the platform auto-scores for the lab (tasks 1–8) are in.** You do not touch these — but read §"Lab answer key" once, because D1–D3 reference what the intern found there.
- 2. Grade D1 (Findings catalogue) first.** It calibrates your eye for "proved it vs asserted it," and D2/D3 build on it. **Check the decoy verdict against the boxed notice in §D1.**
- 3. Grade D2 (Exploit chain + CVSS).** Check each hop has a verbatim PoC and reconciles to a capture line; check the CVSS vectors are *defended*, not pasted.
- 4. Grade D3 (the report) — the heaviest, 40 pts.** Read it the way Bayo would: could an engineer act on it, and is the remediation order argued by effort, not just severity?
- 5. Grade D4 (Ethics) last.** Short, but the highest AI-suspicion deliverable — read for a real decision in the intern's own voice.
- 6. Open the grading page** at `/admin/grading/STAGE_2/<internCode>` and enter your per-deliverable scores.
- 7. Write a 50–100 word grader note.** Required. One strength, one weakness, the deliverable you re-read first.
- 8. Submit.** Your score is hidden from the second grader until they submit too. A large disagreement triggers a super-admin tie-break — that is normal.

The scale — use it on every scoring row

Score each row against three bands, then add the rows to the deliverable's points:

Band	Meaning
Full	Correct, reasoned, and tied to the specific evidence line/artefact
~Half	Core is right but shallow, partly wrong, or thinly justified
0–25%	Missing, copied, off-topic, or fundamentally wrong

Proof beats the right label. A correct vulnerability name with no evidence line scores *lower* than a slightly-mislabelled finding that quotes the exact line proving it. Naming things is cheap; the marks are for evidence and prioritisation.

Universal grading rule

Every finding must be tied to **both** (a) an evidence artefact — a file + line, a capture timestamp, a token, or a payload — **and** (b) the correct class: an **OWASP 2021 category and a CWE id**. A finding with neither earns zero for that row. Evidence-only or class-only = partial. **Full credit needs both.** Made-up CWE/OWASP ids or fabricated capture lines score zero for that row — spot-check one or two per submission against the answer key.

AI-use penalty

If you reasonably believe a deliverable was produced by an LLM, **tick the amber "Flag for suspected AI generation" box** and write 2–3 sentences naming the specific tell. You do **not** apply the penalty yourself: two graders must agree plus super-admin sign-off. Stage 2 tells to watch for:

- Findings named correctly but with **no quoted line** (WHERE username = ' ' . \$username . ' ', "alg": "none", <!ENTITY xxe SYSTEM "file:///etc/passwd">) — the model knows the concept but never read the evidence.
- A CVSS **base score with no vector**, or a vector that does not match the stated metrics (e.g. claims "network, no auth" but writes PR:H).
- A D4 ethics stance that could fit any cohort — no reference to *this* breach, the journalist, the colleague, or the

intern's own findings.

- Confident **business impact with round invented numbers** that don't trace to 05-exfil-sample.csv or the 312-row figure.

Strong writing alone is **not** evidence of AI.

Lab answer key (auto-graded — read once, do not re-grade)

You don't score these, but D1–D3 assume the intern did them. Flags are **per-intern, HMAC-derived, and validated by the platform** — there is no single flag to eye-check. What matters to you is the *concept* each task establishes, because the capstone reuses it.

- **Task 1 — First Contact (recon).** Which exposed service handed The Griot its first *live* data, proven by the **timeline** (first live query timestamp vs the moment the truncated scan was cut off), not by the README banner (which points the wrong way on purpose). A decoy service looks exploitable but post-dates the exfil.
- **Task 2 — The Directory That Answered (log correlation).** Identify the attacker by **behaviour** — the tool user-agent and what it actually retrieved — not "the first IP to touch /legacy-admin/." Planted decoy tokens and a second, legitimate admin IP punish careless attribution.
- **Task 3 — The Query That Should Not Exist (SQL injection).** A **UNION-based** extraction with the correct column count, server-verified. A boolean auth-bypass (' OR '1'='1) logs in but does **not** extract the row — that is the seductive wrong answer.
- **Task 4 — What the Search Field Returned (reflected XSS).** Reflected, in an **HTML attribute** context — requires an attribute breakout + event handler, not a bare <script>. Class is confirmed here, written up later.
- **Task 5 — The Note That Watched Everyone (stored XSS).** The stored cousin of task 4, rendered in a full HTML body context in the admin queue. Different blast radius; do not let interns conflate it with the reflected one.
- **Task 6 — The Request the Server Made (SSRF).** Reach the **exact IAM credentials path** on the link-local metadata service and name the role — reaching /latest/meta-data/ inventory is not enough. Mitigation is IMDSv2 + egress controls. The 04-import-xxe.xml artefact is a second abuse of the same server-side-fetch feature.
- **Tasks 7 & 8 — MCQ.** ATT&CK mapping of the chain, and root-cause vs symptom for the fix. Auto-scored.

(Exact flag/answer values are validated server-side and vary per intern; if you need a specific canonical value confirmed, ask the programme lead.)

The evidence pack (what the capstone is graded against)

D1–D3 are built from the five files in the intern's capstone folder. Here is what each proves, so you can check citations:

- ``01-legacy-admin-login.php`` — recovered login source. Contains, provably: SQL built by **string concatenation** (WHERE username = '' . \$username . '', lines ~64–67); **MD5, unsalted** password hashing (same query); a hand-rolled JWT scheme where **alg:none** is **honoured** (returns the payload with no signature check, ~lines 48–50); a **hardcoded HS256 secret** sankofa-legacy-admin-2019 (~lines 30/38); tokens issued with **no `exp`/`nbf`** (good forever, ~lines 74–75); the session cookie set with **no `Secure`/`SameSite`** (~line 86); and an **open redirect** via ?next= (~lines 88–90).
- ``02-attacker-http-capture.txt`` — the 02:14–02:31 UTC session. Proves the live chain: 02:15:31 SQLi auth bypass (admin' OR '1'='1' 302 to dashboard); 02:16:04 users.php returns **312 user rows**; 02:18:47 **stored XSS** posted to comments/post (cookie-beacon to 185.220.101.9:8443); 02:21:13 **open redirect** honoured; 02:24:02 files/?path=/etc/passwd!' body begins **oot:x:0:0: (path traversal)**. Source IP throughout: 185.220.101.9.
- ``03-legacy-admin-tokens.txt`` — three JWTs. **Token 1** HS256, {"uid":1,"role":"super_admin"}, **no**

exp. Token 2 ``alg:none`, {"uid":19, "role":"admin", "exp":9999999999}`, empty signature. **Token 3** HS256, intern O. adegoke, role:intern. Together they prove: signature can be skipped (`alg:none`), the secret is guessable/hardcoded (so HS256 tokens can be forged), and tokens never expire.

- **`04-import-xxe.xml`** — `<!ENTITY xxe SYSTEM "file:///etc/passwd">` referenced in a `<notes>` field, submitted to `/legacy-admin/import.php` (WAF event 02:23:41Z). **XXE !' local file read** Same `/etc/passwd` target as the `files/?path=` request in the capture — two routes to one outcome.
- **`05-exfil-sample.csv`** — 20 placeholder rows preserving field shape (customer_id, name, email, masked account no., **balance in NGN**, last login). Stands in for the **312-row** real export. Balances range to ~'950,000; use it to quantify impact.

Hand-graded capstone — full answer keys

D1 — Findings catalogue (and the decoy) — 30 points

The intern catalogues every weakness they can substantiate from the evidence pack — one row each (description · OWASP+CWE · exact evidence line · confidence) — then handles one decoy. Nobody told them how many findings there are.

Answer key — the substantiated findings (a strong catalogue names most of these):

#	Finding	OWASP 2021 · CWE	Proof
1	SQL injection in the login query (string concatenation)	A03 Injection · CWE-89	<code>`01-...login.php` ~64-67;`02-...capture` `02:15:31`</code>
2	Unsalted MD5 password hashing	A02 Cryptographic Failures · CWE-916 (+CWE-759)	<code>`01-...login.php` ~66</code>
3	JWT <code>`alg:none`</code> accepted (no signature check)	A07 Identification & Auth Failures · CWE-347	<code>`01-...login.php` ~48-50;`03-tokens` Token 2</code>
4	Hardcoded / guessable JWT signing secret	A02 / A05 · CWE-798 (+CWE-321)	<code>`01-...login.php` ~30/38</code>
5	Tokens have no expiry (<code>`exp`/`nbf`</code>)	A07 · CWE-613	<code>`01-...login.php` ~74-75;`03-tokens` Token 1</code>
6	Stored XSS in comments/notes	A03 Injection · CWE-79	<code>`02-...capture` `02:18:47`</code>
7	XXE in the XML import (external entity !' file read)	A05 Security Misconfiguration · CWE-611	<code>`04-import-xxe.xml`</code>
8	Path traversal / arbitrary file read	A01 Broken Access Control · CWE-22	<code>`02-...capture` `02:24:02`</code>
9	Open redirect via <code>`?next=`</code>	A01 · CWE-601	<code>`01-...login.php` ~88-90;`02-...capture` `02:21:13`</code>
10	Session cookie without <code>`Secure`/`SameSite`</code>	A05 · CWE-614 (+CWE-1275)	<code>`01-...login.php` ~86</code>

Findings 1–7 are the substantial ones; 8–10 are the marks that separate a thorough catalogue from a basic one. Accept correct class variants that are defensible (e.g. XXE under A05 or the legacy A04:2017; cookie flags under A05 or A07).

& p The decoy — *CONFIRM the canonical answer with the programme lead before grading*

D1 requires the intern to identify one construct that **looks** hand-rolled and dangerous and decide whether it is actually exploitable. The strongest decoy candidate in the source is the **hand-rolled crypto that is in fact implemented safely**: the `b64url_encode/b64url_decode` helpers (`base64url` is encoding, not a weakness) and, critically, the **HS256 verification branch that uses `hash_equals()`** — a correct, constant-time comparison. A careless analyst flags "hand-rolled JWT crypto = Critical"; the disciplined one shows the exploitable flaws are `alg:none` acceptance and the hardcoded secret, **not** the signature-compare routine, which is sound.

Full credit (decoy): identifies a tempting hand-rolled construct and proves it safe at the mechanism level (e.g. "`hash_equals` is constant-time; this branch is fine — the bug is `alg:none` above it"), OR shows a genuine exploit if they argue it is exploitable.

Zero / AI tell: flags a safe construct Critical with no mechanism, or invents an exploit for it.

This guide infers the decoy from the source; if the programme scripted a specific intended decoy, confirm it and replace this box before grading a batch.

Scoring (30 pts):

Row	Pts	Full credit
Findings 1–5 catalogued (desc + OWASP + CWE + quoted evidence line)	12	~2–3 each; class **and** evidence line both correct
Findings 6–7 (stored XSS, XXE) catalogued with evidence	5	Quotes the capture line / the XML entity declaration
Coverage of the access-control set (8–10)	4	At least two of path traversal / open redirect / cookie flags, evidenced
Method section — how they read the evidence and what counts as a finding	3	Specific to this pack; "trust the code, not the comments" applied
The decoy — identified + verdict proved either way	6	See box above

Red flags: a catalogue that stops at the SQL injection (caps coverage rows); classes with no evidence line; a **fabricated evidence line** (zero that row, check the AI flag); flagging the safe hand-rolled crypto Critical.

D2 — Exploit chain + CVSS and business impact — 35 points

The intern rebuilds the attacker's path as one chain, gives every hop a verbatim PoC reconciled to a capture line, scores each finding with a CVSS 3.1 **vector**, and quantifies impact.

Answer key — the canonical chain (from ``02-...capture``):

- Recon / first contact** — `02:14:08-02:14:23 GET /legacy-admin/ and login.php.`
- Failed guess** — `02:14:55 admin/password !' "Invalid credentials"` (shows they tried the easy path first).
- SQL injection auth bypass** — `02:15:31 username=admin' OR '1'='1 !' 302 to dashboard.php,` session issued. *Hands the attacker an authenticated admin session.*
- Bulk PII access** — `02:16:04 users.php !312 rows` (username + email + role).
- Stored XSS** — `02:18:47 comments/post cookie-beacon` — persists, fires for every admin who views the queue.
- Open redirect** — `02:21:13 ?next=` honoured to an external host.
- File read** — `02:24:02 files/?path=/etc/passwd !root:x:0:0:` (and the parallel XXE route via `import.php, 04-import-xxe.xml, WAF 02:23:41Z`).

- 8. Exfil** — terminates at the 312-row customer export (05-exfil-sample.csv shape). The **JWT flaws** (alg:none / hardcoded secret / no-exp, from 03-tokens) let a forged super_admin token persist access and reach the /api/v1/internal service the cookie is shared with — a scope escalation beyond the legacy app.

Expected CVSS vectors (accept defensible variants — the vector + justification is what's graded, not digit-matching):

Finding	Suggested CVSS 3.1 vector	~Score
SQL injection (auth bypass !' PII read)	`AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N`	9.1 Crit
Broken JWT auth (alg:none + hardcoded secret + no-exp; token honoured by internal API)	`AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:H/A:N`	9.x–10 Crit
XXE !' local file read	`AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N`	~7–8 High
Stored XSS (admin session theft)	`AV:N/AC:L/PR:L/UI:R/S:C/C:H/I:L/A:N`	~8 High
Path traversal	`AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N`	~6.5 Med/High

The ****scope change (S:C)**** on the JWT finding is the senior insight — the legacy-admin token is honoured by a different security authority (the internal microservice). Reward interns who argue it; do not penalise a defended `S:U`.

Business impact: 312 customers, PII + financial (names, emails, account numbers, balances). Using 05-exfil-sample.csv, balances run into the hundreds of thousands of naira each (sample max ~'950,000) — a defensible exposure estimate is in the **tens of millions of naira** plus NDPA breach exposure. Tie it to **BoltCash launching in ~6 weeks** (the risk is now, not someday). Reward stated assumptions over a confident bare number.

Scoring (35 pts):

Row	Pts	Full credit
Chain reconstructed in order, each hop reconciled to a capture line	12	Hops 3–8 present and in order, each citing the timestamp/line
Verbatim PoC per hop (request / forged token shown as header.payload.sig / XML body)	8	Artefacts shown, not described
CVSS — complete vector string per finding, metrics justified	8	Vectors present + defended; bare numbers cap this at half
Business impact in customers and naira, assumptions stated	7	312 records + a naira estimate traceable to the exfil sample + BoltCash timing

Red flags: prose descriptions instead of PoCs; a chain with gaps or out of order; CVSS numbers with no vector (cap at half); "significant impact" with no number.

D3 — The report Bayo acts on (+ detection stopgaps) — 40 points

The finished 6–8 page pentest report. It folds in D1/D2, but is graded on whether an **engineer can act on it** and whether the **remediation order is argued by risk reduction per hour of effort**, not severity alone.

What a top report contains:

- Executive summary** a CFO understands, ending with the headline: ~312 customers' PII/financial data at risk, tens of millions of naira, BoltCash in 6 weeks. No jargon, no CWE ids here.

- **Threat model** — unauthenticated, internet-reachable 2019 app sharing auth with an internal service; realistic attacker is the one in the capture.
- **Findings** tied to evidence (the D1 set), ordered by remediation priority, each with its CVSS from D2.
- **The disproved decoy**, stated plainly (buys trust, stops a wasted sprint).
- **Exploit chain** in report form, noting the single fix that breaks it earliest (parameterising the login query / fixing auth kills the chain at hop 3).
- **Remediation order argued by risk-reduction-per-hour**. Example of the reasoning we want: *parameterised query for the SQLi is a few hours and breaks the chain at the first hop !' rank 1, above a higher-CVSS fix that takes weeks*. Disabling external entities in the XML parser and enforcing an algorithm allow-list (no `alg:none`) + rotating the secret are similarly cheap, high-value.
- **Two detection stopgaps** for the two most dangerous findings — each a concrete SIEM/log query or WAF rule naming the **exact field/pattern** and an **expected false-positive rate**. Good examples: alert on inbound JWTs whose decoded header contains `"alg": "none"`; WAF rule on login POST bodies matching an OR `'1' = '1'` style tautology; alert on any import body containing a `SYSTEM` external-entity declaration.

Scoring (40 pts):

Row	Pts	Full credit
Executive summary — jargon-free, ends with customers-and-naira headline	6	A CFO could act on it
Threat model	4	Tight, specific to this app and attacker
Findings tied to evidence (carries D1, engineering-readable)	8	Each finding has its proof line + severity
Disproved decoy stated	3	Named, with the one-line reason it is safe
Exploit chain in report form + the earliest-break fix	5	Shows it is one path, not isolated bugs
Remediation order argued by risk-reduction-per-hour	8	Order defended by effort vs risk, **not** CVSS order alone
Two detection stopgaps (exact field/pattern + FP rate)	6	Deployable; names the field; gives an FP estimate

Red flags: a report that just re-lists findings by CVSS and calls that a remediation plan (cap the remediation row at half — the brief explicitly rewards effort-based prioritisation); "add monitoring" with no concrete rule; an executive summary full of CWE numbers; any finding not tied to evidence (cut it, as the brief instructs).

D4 — Ethics stance — 15 points

300–500 words, the intern's own voice. The setup: the breach is public (TechCabal ran it), a journalist the intern knows personally asks for "background, off the record," and a senior colleague is about to take the fall while the intern holds the timeline proving who knew what. PII has confirmed left the building.

What full credit looks like: a *real decision*, not a balanced essay — they answer all four prompts (what they tell the journalist and why; what they will/won't put in writing; how they handle evidence that could protect or implicate the colleague; the one action they take regardless of their manager), they cite **at least one ISC2 Code of Ethics canon by number** and make it load-bearing, they name the **NDPA breach-notification duty** (notify the NDPC, and affected data subjects where high risk — tied to the 312 customers), and at least one sentence is **anchored to their own Stage 2 work**.

ISC2 canons, for your reference (interns must cite by number): (I) Protect society, the common good, public trust, and the infrastructure; (II) Act honourably, honestly, justly, responsibly, legally; (III) Provide diligent and competent service to principals; (IV) Advance and protect the profession.

Scoring (15 pts):

Row	Pts	Full credit
Answers all four prompts with a clear, owned decision	6	Journalist + in-writing + colleague + the non-negotiable action
ISC2 canon cited by number and load-bearing	3	Drives the reasoning, not name-dropped
NDPA breach-notification duty, tied to confirmed PII	3	Names the duty + connects it to the 312 records
Own voice + anchored to their own findings (anti-AI)	3	A sentence only this intern could have written

Red flags: a generic ethics essay that fits any cohort or any breach (strongest AI tell in this stage — flag it); reciting policy with no decision; no canon number; no NDPA. Two graders read every D4 independently; if both find it generic or untraceable to the intern's own work, treat it as not submitted.

Grader checklist (every Stage 2 submission)

1. Confirm the lab (tasks 1–8) already has platform scores — do not re-grade them.
2. Hand-grade the capstone D1–D4 using the answer keys and scoring tables above (120 pts).
3. **Confirm the canonical decoy** (§D1 box) with the programme lead before grading a batch.
4. Every finding needs **both** an evidence line and a correct OWASP+CWE — spot-check one for a fabricated line or id.
5. Score as a percentage: $\text{points earned} \div 120 \times 100$. The cutoff (set by programme staff) is applied to that percentage.
6. Leave a one-line justification per deliverable and a 50–100 word grader note.
7. Flag integrity/AI concerns to a PM with a specific reason; never eliminate on suspicion alone.
8. Submit by the stage deadline; your score is hidden from the second grader until they submit.